

Звіт про десктоп-клієнт проєкту Remote File Folder Client (спрощений аналог Google Drive)

Уточкіна Яна

Етап: десктоп-клієнт (macOS/SwiftUI) – проєктування,
використання та тестування
30 вересня 2026 р.

Анотація

Цей звіт документує десктоп-клієнт macOS проєкту **Remote File Folder Client** (desktop/Sources/FileSpaceDesktop/) – спрощеного аналога Google Drive з веб- і десктоп-клієнтами, Go-бекендом, PostgreSQL та синхронізацією локальної теки. У звіті описано внутрішню структуру клієнта, розглянуто його прецеденти використання зі скріншотами та задокументовано покриття модульними тестами, включно з поведінкою сортування й фільтрації (сортування за “*Edited by*”, фільтрація за `.cs/.jpg`).

Зміст

1 Вступ	2
2 Архітектура десктоп-клієнта	2
2.1 Внутрішня структура	2
2.2 Синхронізація	3
3 Прецеденти використання та скріншоти	3
3.1 Реєстрація та автентифікація	3
3.2 Перегляд файлів: атрибути, стовпці, сортування, фільтрація . .	4
3.3 Перегляд вмісту файлу	6
3.4 Вивантаження, завантаження, видалення	7
3.5 Синхронізація локальної теки	9

4 Модульне тестування	10
4.1 Налаштування тестів	10
4.2 Покриття за областями	10
4.3 Покриття сортування та фільтрації	11
5 Висновки	12

1. Вступ

Десктоп-клієнт – один із двох фронтендів (другий – веб-клієнт на React) до єдиного Go API та бази PostgreSQL, самостійно розгорнутих на домашньому Ubuntu-сервері за Cloudflare Tunnel. Усі прецеденти, які система підтримує через будь-який клієнт – реєстрація, автентифікація, перегляд файлів із сортованими/фільтрованими атрибутами, перегляд вмісту `.java/.png`, вивантаження, завантаження, видалення – доступні з десктоп-клієнта; крім того, він є *єдиним* клієнтом, що реалізує безперервну фонову синхронізацію локальної теки, оскільки браузер не може постійно спостерігати за локальною текою.

2. Архітектура десктоп-клієнта

2.1 Внутрішня структура

Тека з вихідним кодом десктоп-клієнта, `desktop/Sources/FileSpaceDesktop/`, групує код так: SwiftUI-подання відокремлені від рушія синхронізації (`DirectoryWatcher`, `SyncViewModel` та `SyncState/FolderScanner` з `Manifest.swift`) і від спільного мережевого шару `APIClient/KeychainStore`:

- **Подання (Views):** `LoginView`, `MainView` (таблиця файлів, сортування/фільтрація/видимість стовпців), `FilePreviewView` (`.java/.png`), `SyncPanelView` та `SyncConflictsSectionView` (керування синхронізацією теки та інтерфейс розв'язання конфліктів), `ContentView` (верхньорівнева навігація між станами “не увійшов”/“увійшов”);
- **View-моделі:** `FilesViewModel` (список файлів, сортування/фільтрація, вивантаження/завантаження/видалення), `SyncViewModel` (цикл узгодження, виявлення/розв'язання конфліктів), `AppState` (стан сесії/автентифікації);
- **Мережа та сховище:** `APIClient` (REST-клієнт на `URLSession`, за протоколом `APIClientProtocol`, який використовується для тестування), `KeychainStore` (зберігання токенів через macOS Keychain), `Models.swift` (Codable DTO, що відповідають JSON-формату Go API);
- **Рушій синхронізації:** `DirectoryWatcher` (обгортка `FSEvents`), `Manifest.swift` (`FolderScanner`, `SyncState`, `SyncedFileEntry`).

Десктоп-клієнт звертається до Go API-сервера лише через HTTPS за публічним хостнеймом Cloudflare Tunnel – тим самим, що використовує браузерний JavaScript веб-клієнта – проходячи через серверні шари `Handler` → `Service` → `Repository` та, для вмісту файлів, інтерфейс `FileStorage`.

2.2 Синхронізація

Синхронізація – найскладніша функція десктоп-клієнта і єдина, що не має аналога у веб-клієнті. За кожним тригером – сповіщенням про зміну від FSEvents через DirectoryWatcher або періодичним опитуванням – SyncViewModel викликає POST /api/sync/diff, надсилаючи lastSyncedVersion та маніфест (ім'я, розмір, sha256), обчислений FolderScanner; сервер повертає дії download/delete_local/conflict разом із newVersion. Клієнт застосовує ці дії, а потім незалежно сканує свою локальну теку й вивантажує, повторно вивантажує або видаляє файли на основі власного збереженого SyncState.

Розв'язання конфліктів

Дія conflict означає, що той самий файл змінився з обох боків від часу останньої синхронізації: сервер порівнює наданий клієнтом маніфест (ім'я, розмір, sha256) із власним записом і повертає дію conflict із розміром, часом модифікації та останнім редактором віддаленої сторони, коли хеші двох сторін справді не збігаються. На десктоп-клієнті SyncViewModel записує такі дії в опублікований список conflicts замість автоматичного розв'язання, а SyncConflictsSectionView (частина вже наявної панелі синхронізації) дозволяє користувачу обрати **Keep Local** (повторно вивантажити, перезаписавши віддалену копію), **Keep Remote** (перезаписати локальний файл) або **Keep Both** (зберегти локальний файл під перейменованою копією й прийняти віддалену версію).

3. Прецеденти використання та скріншоти

У цьому розділі розглянуто прецеденти використання десктоп-клієнта, кожен зі скріншотом працюючого застосунку.

3.1 Реєстрація та автентифікація

Реєстрація (рис. 1) створює новий обліковий запис за логіном/паролем; *автентифікація* (рис. 2) потрібна для кожної іншої файлової операції, оскільки робоча область доступна лише після успішного входу, після чого APIClient зберігає отриману пару JWT access/refresh у macOS Keychain через KeychainStore.

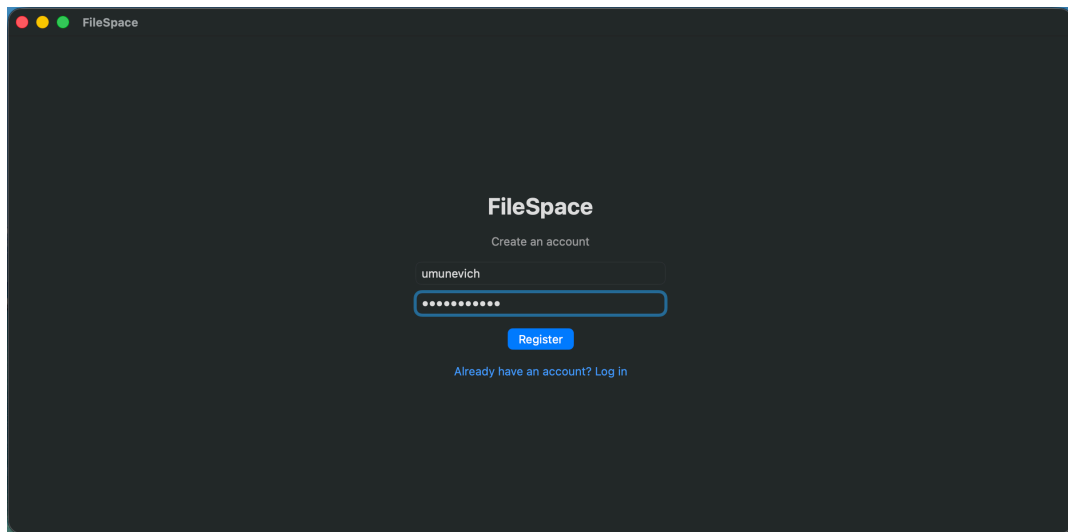


Рис. 1: Екран реєстрації

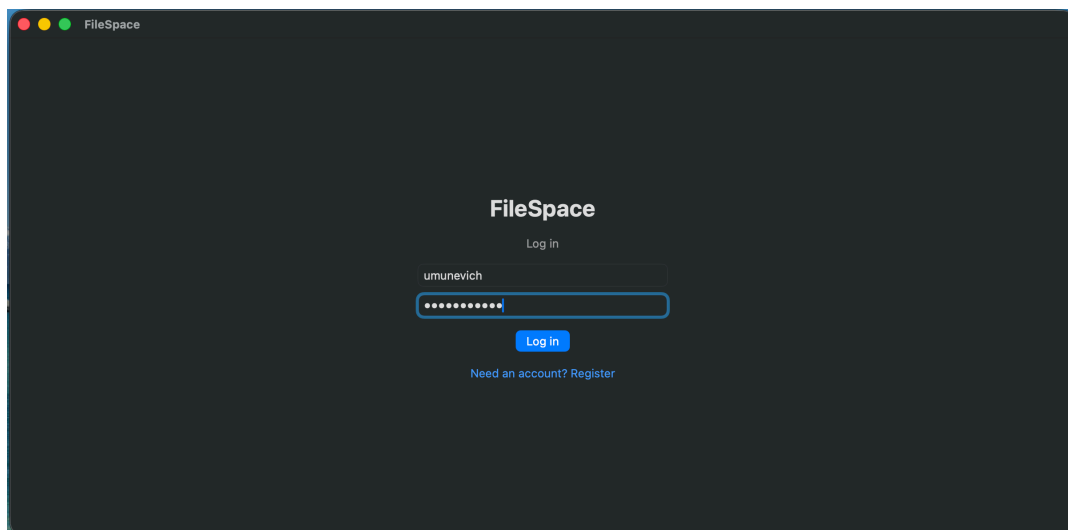


Рис. 2: Екран входу

3.2 Перегляд файлів: атрибути, стовпці, сортування, фільтрація

Рис. 3 показує основну таблицю файлів з усіма видимими стовпцями атрибутів (ім'я, створено, змінено, автор вивантаження, автор редагування); стовпець імені приховати не можна, тоді як інші перемикаються через меню "Columns" (рис. 4).

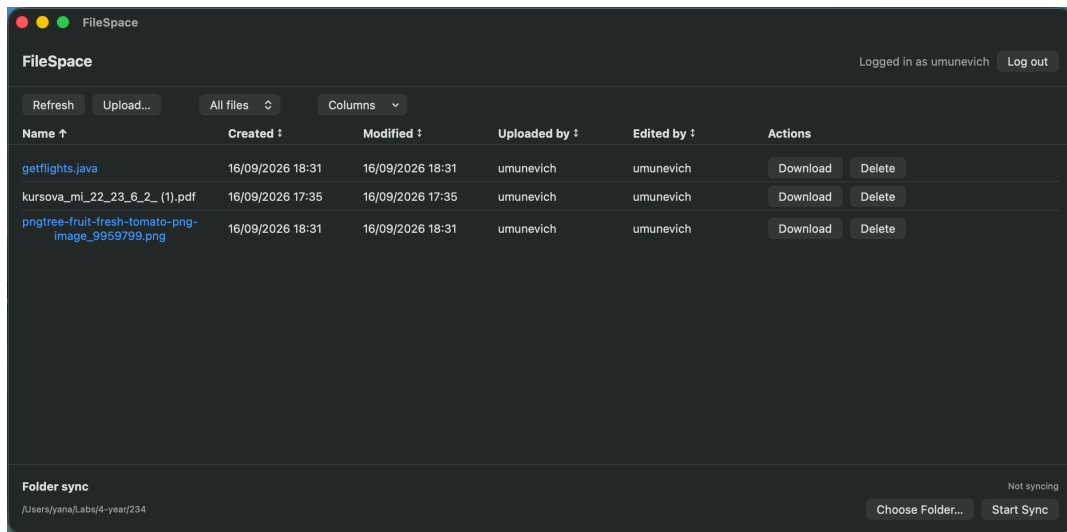


Рис. 3: Основний список файлів з усіма видимими стовпцями атрибутів

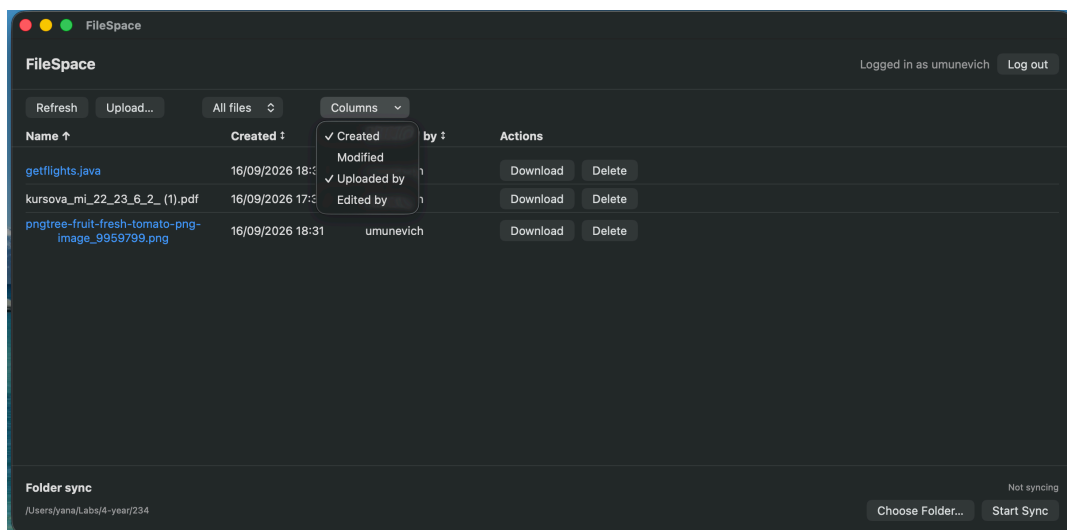


Рис. 4: Меню Columns: перемикання стовпця, відмінного від імені

Сортування: список файлів сортується за “*Edited by*” за зростанням або спаданням при кліку на заголовок цього стовпця, що циклічно перемикає `FilesViewModel.sortDirection` через `none` → `ascending` → `descending` → `none` (рис. 5).

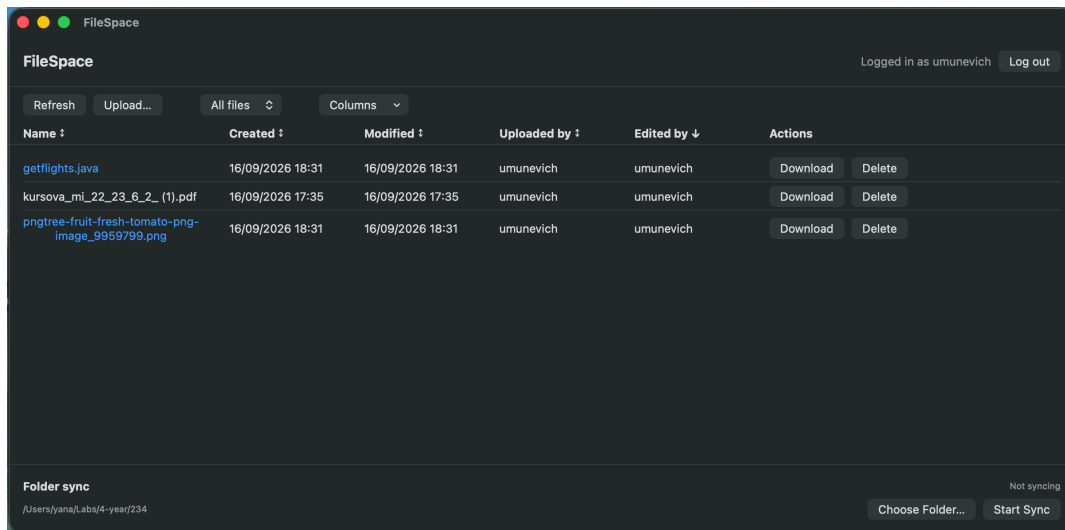


Рис. 5: Список файлів, відсортований за “Edited by”

Фільтрація: випадний список розширень, який формується з розширень, що реально присутні серед файлів користувача, фільтрує список лише до .cs або лише до .jpg (або назад до “All”) через `FilesViewModel.extensionFilter` (рис. 6).

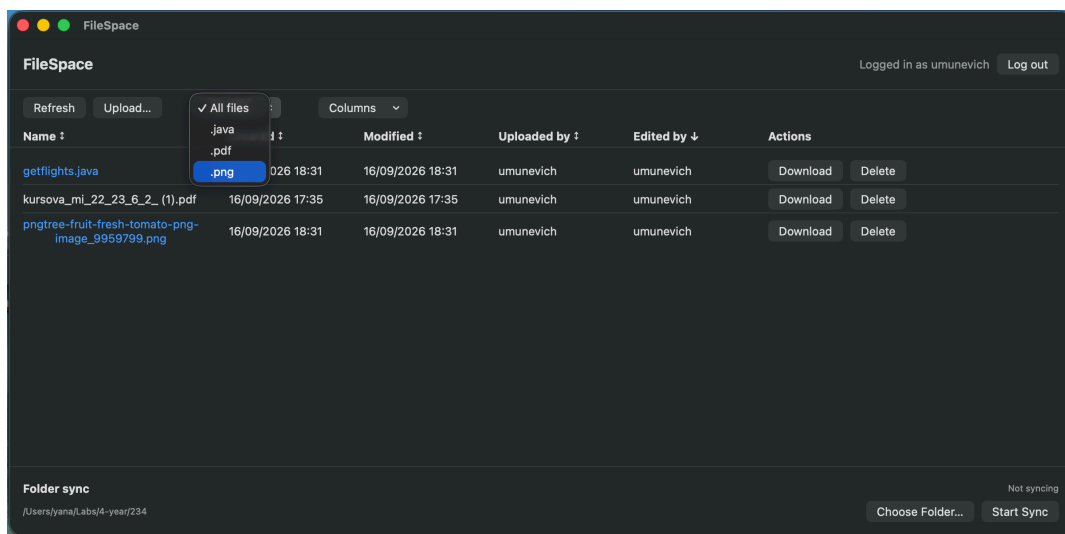


Рис. 6: Застосований фільтр за розширенням (.cs або .jpg)

3.3 Перегляд вмісту файлу

Клік на файл показує його вміст, якщо розширення - .java (відображається як текст, рис. 7) або .png (відображається як зображення, рис. 8); файли з будь-яким іншим розширенням можна переглядати у списку й завантажувати, але не відображати їхній вміст.

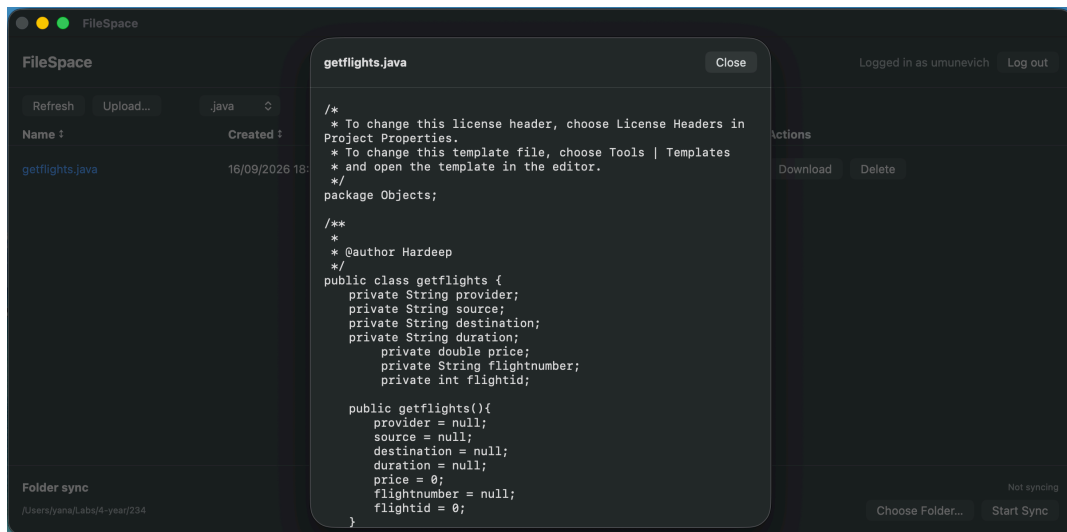


Рис. 7: Файл .java, показаний як текст

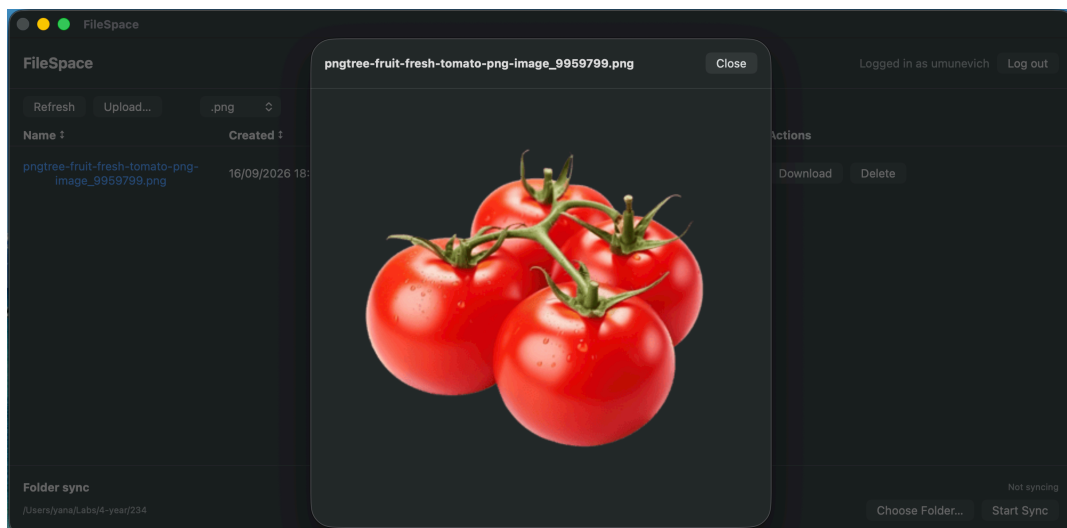


Рис. 8: Файл .png, показаний як зображення

3.4 Вивантаження, завантаження, видалення

Рис. 9 показує вивантаження в процесі – великі файли розбиваються на chunk'и, щоб жоден окремий HTTP-запит не перевищував ліміт розміру тіла безкоштовного тарифу Cloudflare Tunnel, а `uploadProgress` відображає кількість надісланих байтів по кожному chunk'у. Рис. 10 та 11 показують завантаження файлу в теку Downloads користувача та видалення файлу з віддаленої робочої області.

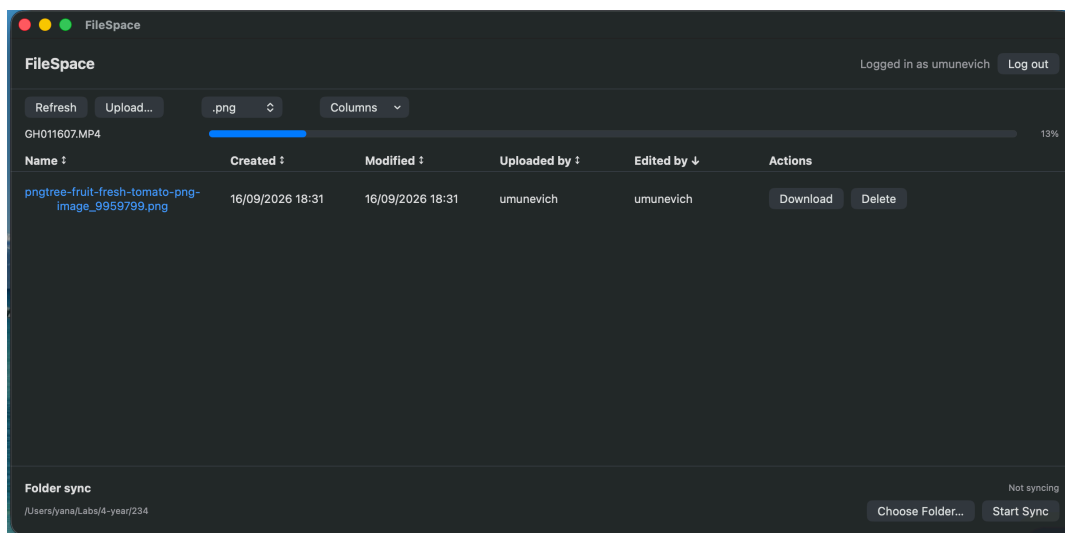


Рис. 9: Вивантаження файлу в процесі

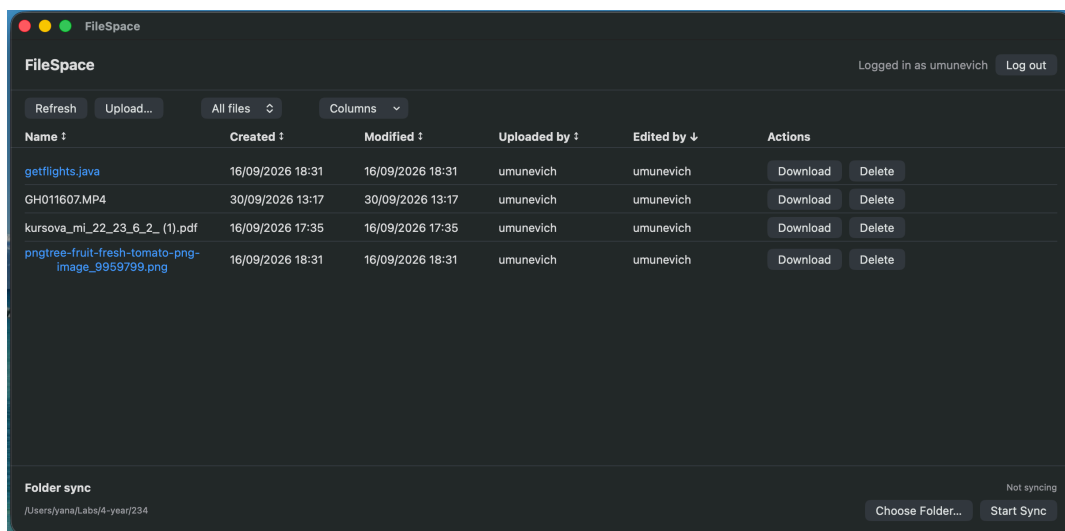


Рис. 10: Завантаження файлу

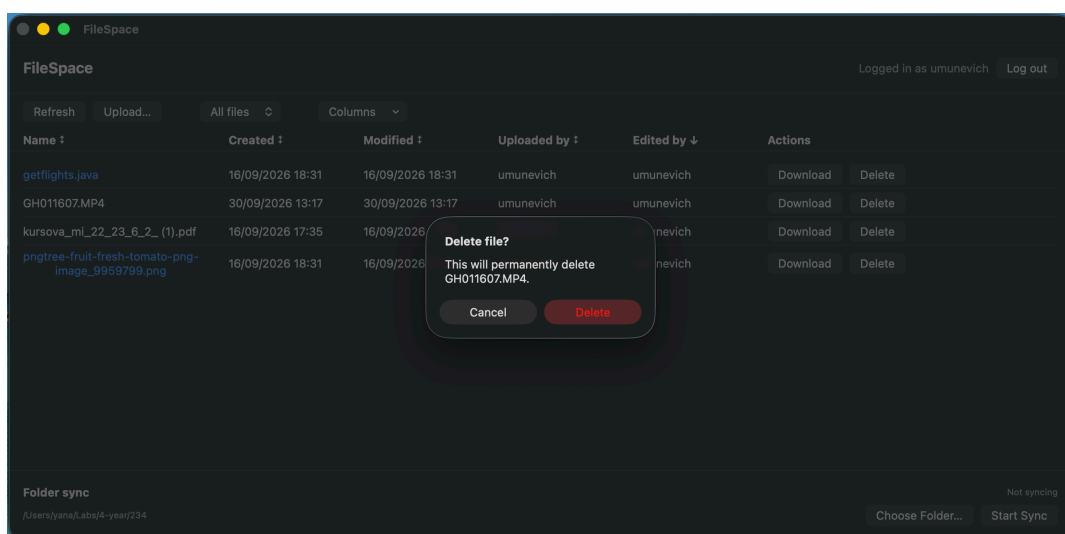


Рис. 11: Видалення файлу

3.5 Синхронізація локальної теки

Рис. 12 показує панель синхронізації з обраною текою й активною синхронізацією: DirectoryWatcher генерує сповіщення про зміни на основі FSEvents, а SyncViewModel також періодично опитує сервер, виконуючи цикл узгодження, описаний у §2.2. Рис. 13 показує ту саму панель, коли сервер повідомляє про справжній конфлікт: файл показано разом із його локальним/віддаленим розміром та редактором, а також діями Keep Local / Keep Remote / Keep Both.

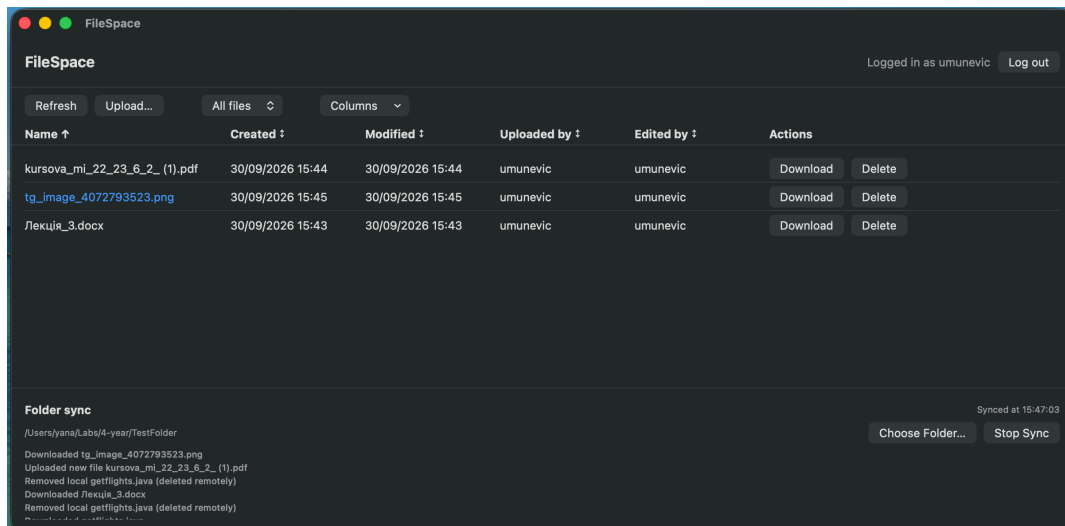


Рис. 12: Панель синхронізації теки, активна синхронізація

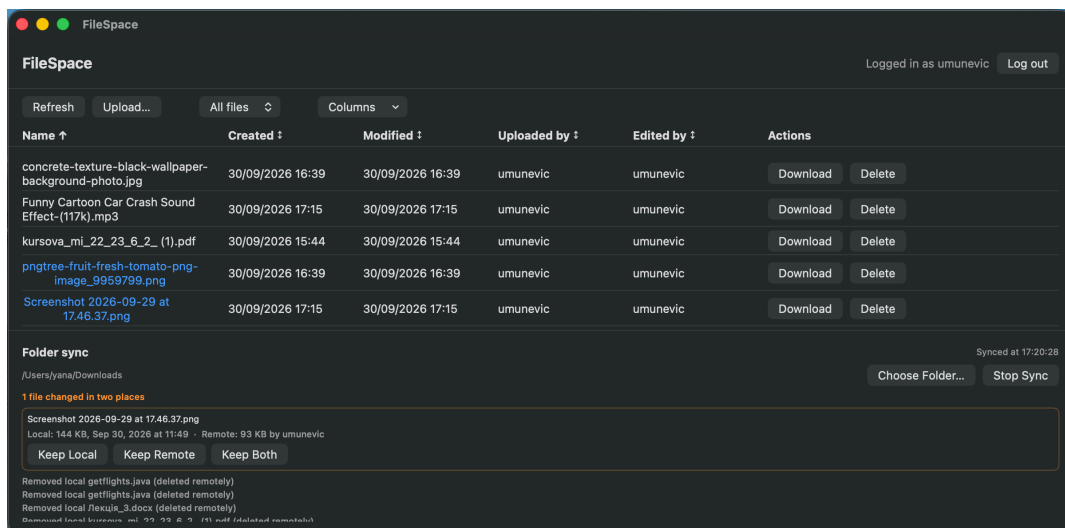


Рис. 13: Панель синхронізації теки з виявленим конфліктом та варіантами його розв'язання

4. Модульне тестування

4.1 Налаштування тестів

Тестова ціль десктоп-клієнта, `FileSpaceDesktopTests`, використовує `XCTest` (запускається через `swift test` з `desktop/` або через `make desktop-test`), а не новіший модуль `swift-testing`, оскільки в цьому середовищі встановлено лише `Command Line Tools` від `Xcode`, які не постачають модуль `Testing`. Мережеві або залежні від ОС співучасники замінюються `in-memory` заглушками, тож набір тестів виконується офлайн і детерміновано:

- `SyncViewModel` і `FilesViewModel` залежать від `APIClientProtocol`, а не від конкретного `APIClient`, тож тести підставляють безмережевий `FakeAPIClient` замість `URLSession`;
- тести `FolderScanner` і стану синхронізації використовують реальні тимчасові каталоги на диску, а не заглушки файлової системи, оскільки код, що тестується, *сам є* взаємодією з файловою системою;
- `KeychainStoreTests` працюють із реальним `macOS Keychain` (`in-memory` заміни для нього немає), використовуючи виділені ключі, які очищаються в `tearDown`.

4.2 Покриття за областями

Таблиця 1 підсумовує набір модульних тестів.

Табл. 1: Набори модульних тестів у `desktop/Tests/` `FileSpaceDesktopTests/`

Клас тестів	К-сть	Що покриває
<code>FileRecordDecoding-Tests</code>	2	Декодування <code>FileRecord</code> з точного JSON-формату бекенду, включно з перейменуванням ключа <code>extension</code> → <code>extensionName</code>
<code>SyncStateTests</code>	1	Round-trip <code>SyncState/SyncedFileEntry</code> через кодування у JSON
<code>FolderScannerTests</code>	4	Сканування локальної теки: обчислення хешу, пропуск прихованих файлів/підкаталогів та коректне розрізнення “неможливо прочитати” від “не існує”
<code>PreviewSupportTests</code>	1	Лише <code>.java/.png</code> вважаються придатними для перегляду
<code>KeychainStoreTests</code>	5	Встановлення/читання/перезапис/видалення токенів у реальному <code>macOS Keychain</code> з ізоляцією ключів

Табл. 1: Набори модульних тестів у desktop/Tests/FileSpaceDesktopTests/

Клас тестів	К-сть	Що покриває
APIClientTests	6	Декодування запитів/відповідей, порядок і прогрес chunked-вивантаження, вибір між одноразовим і chunked-вивантаженням, поведінка повтору запиту після 401 з оновленням токена
SyncViewModelTests	11	Цикл узгодження: обробка нових/змінених/локально видалених файлів, надсилання маніфесту, виявлення/фіксація конфліктів та всі три способи розв'язання (Keep Local/Remote/Both)
FilesViewModelTests	7	Сортування за "Edited by" (за зростанням, за спаданням, без урахування реєстру) та фільтрація за розширенням (.cs, .jpg, без фільтра), включно з одночасним застосуванням сортування й фільтра
Разом	37	

4.3 Покриття сортування та фільтрації

Список файлів сортується за "Edited by" (за зростанням і за спаданням) та фільтрується до **всіх файлів**, лише файлів .cs або лише файлів .jpg. Ця логіка міститься в FilesViewModel.displayedFiles, toggleSort(_:) та extensionFilter і перевіряється тестами з FilesViewModelTests.swift:

```
testSortByEditedByAscending
testSortByEditedByDescending
testSortByEditedByIsCaseInsensitive
testNoFilterShowsAllExtensions
testFilterByCsExtensionShowsOnlyCsFiles
testFilterByJpgExtensionShowsOnlyJpgFiles
testFilterAndSortByEditedByComposeTogether
```

FilesViewModel залежить від APIClientProtocol, а не від конкретного, прив'язаного до мережі APIClient (той самий принцип, що вже використовує SyncViewModel), і саме це дозволяє цим тестам підставляти заглушку та виконуватися офлайн.

```
Test Suite 'All tests' passed at 2026-09-30 10:53:05.
Executed 37 tests, with 0 failures (0 unexpected)
in 0.190 (0.197) seconds
```

5. Висновки

Десктоп-клієнт реалізує кожен прецедент, потрібний проєкту, який нативний застосунок macOS може змістовно додати порівняно з веб-клієнтом: повний перегляд файлів із сортуванням за “Edited by” та фільтрацією за .cs/.jpg, перегляд вмісту .java/.png, вивантаження/завантаження/видалення, а також – унікально серед обох клієнтів – безперервну фонову синхронізацію теки, включно з виявленням конфліктів на основі маніфесту та трибічним (Keep Local/Remote/Both) інтерфейсом розв’язання. З погляду тестування, основні співучасники клієнта (мережа, сканування теки, стан синхронізації, доступ до Keychain, цикл узгодження та сортування/фільтрація списку файлів) мають модульне покриття. Усі 37 тестів набору проходять успішно.